Motion Sensor Based Intrusion Detection and Surveillance System with Smart Alerts

By

Ajwad Shamim

(2021-EE-01)

Mubashir Naeem

(2021-EE-07)

Danyal Farooq

(2021-EE-21)

Session 2021-2025

Department of Electrical Engineering

Faculty of Engineering and Technology

University of Poonch Rawalakot

Azad Jammu and Kashmir

Table of Contents

Abstract ..... **v**

Chapter 1. INTRODUCTION **1**

1.1 Problem Statement 1

1.2 Objectives 1

Chapter 2. MATERIALS and METHODS **2**

2.1 System Design and Architecture 2

2.1.1 Overview of System Goals 2

2.1.2 Hardware and Configuration 2

2.1.3 Logical Tiering and Sensor Fusion Design 3

2.1.4 Data Acquisition and Processing Pipeline 4

2.1.5 Live streaming and Monitoring. 4

2.1.6 On-Device Display and Visual Feedback 5

2.1.7 Alerts and Cloud Incorporation 5

2.1.8 Metadata and Evidentiary Integrity 5

2.1.9 Software Structure 6

2.1.10 Summary 6

2.2 Data Storage and Alert Process 7

2.2.1 Overview 7

2.2.2 Event-Driven Storage 8

2.2.3 Local Storage Management 8

2.2.4 Cloud-Based Storage (Google Drive) 9

2.2.5 Metadata and Evidence Integrity 9

2.2.6 Architecture of Alerts 10

2.2.7 Distinguished Notifications 10

2.2.8 Live Streaming and Monitoring 11

2.2.9 Fail Data Handling 11

2.2.10 Summary of Data Flow 11

Chapter 3. Results and Discussion 14

3.1 Test setup 14

3.2 Theoretically Calculated Continuous Recording 14

3.3 Event-based Recording 15

3.3.1 Calculations 16

3.4 Locating Relevant Evidence 16

3.5 Cloud Upload and Network Usage 17

3.6 Discussions 17

3.6.1 General Motion Event 17

3.6.2 Intrusion Event 17

3.6.3 Observations 17

3.7 Recommendations 18

3.8 Conclusion 18

References 20

Appendix 23

List of Figures

Figure 1: Project Design 7

Figure 9: Flow Chart 13

List of Tables

Table 1: Acronyms iv

Table 3: Test setup 14

Table 4 : Continuous Recording 15

Table 5: File Type and Size 15

Table 6: Event Summary 15

Table 7: Storage Comparison 16

Table 8: Network and Cloud usage Comparison 17

Table 1: Acronyms

Acronym	Expansion
CCTV	Closed Circuit Television
CPU	Central Processing Unit
CSI	Camera Serial Interface
fps	Frames per second
GPIO	General Purpose Input Output
HVAC	Heating, Ventilation and Air Conditioning
I2C	Inter-Integrated Circuit
I/O	Input/Output
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KB	Kilobyte
MB	Megabyte
GB	Gigabyte
MQTT	Message Queuing Telemetry Transport
MQTT/TLS	MQTT over TLS
NIST	National Institute of Standards and Technology
NTP	Network Time Protocol
NoIR	No Infrared
OLED	Organic Light Emitting Diode
RAM	Random Access Memory
RTSP	Real Time Streaming Protocol
NTP	Network Time Protocol
RTC	Real-Time Clock
SD / microSD card	Secure Digital card
SHA-256	Secure Hash Algorithm 256
USB OTG	USB On-The-Go
Wi-Fi	Wireless Fidelity
SDG / SDGs	Sustainable Development Goal(s)

Abstract

Continuous surveillance systems generate massive volumes of useless footage, and overload storage and review procedures. To solve this, we have created a small event-based system that captures only relevant activity and provides tiered alerts. The system is powered by a Raspberry Pi Zero 2W and uses a Raspberry Pi Camera V2 NoIR with various sensors like PIR, ultrasonic, microwave Doppler, and a laser tripwire. This system is based on two zones of detection. The general zone is based upon PIR and ultrasonic confirmation, capturing a timestamped snapshot and sending a 'motion detected' Discord notification. The critical zone will need confirmation of PIR and microwave Doppler sensor, and laser-break confirmation to trigger a buzzer, take a snapshot, record a 20-second clip and transmit it to Google Drive using rclone and an 'intrusion detected' Discord notification. A live stream of low latency is provided for real time monitoring but is not recorded. Each event is stored using ISO timestamps and event-specific JSON manifests; optional checksums (SHA-256) are used to support file integrity.. Experiments and theoretical analysis have demonstrated that recording only those events that are important optimizes storage compared to continuous recording of the same bitrate and that event files are quicker to scan through. The system is functional and is efficient in motion and intrusion detection. Significant limitations include the requirement to adjust sensor timing for noisy conditions, delayed alert caused by the delay in the network, and false alarms in case of changing light. Future work will involve multi-node scaling, enhanced metadata indexing, and lightweight on-device validation to minimize false alerts. The system exhibits an evidence-based practical approach that maximises storage and provides prioritized visual alerts.

Chapter 1. INTRODUCTION

1.1 Problem Statement

Conventional surveillance systems record continuously and lack smart alert mechanisms, leading to poor utilization of storage and false alarms. Hence, there is a need for a smart, event-based surveillance system that incorporates a series of sensing technologies to differentiate between normal motion and verified intrusion. This system must optimize video storage and record video only in the cases of relevant intrusion and use an intelligent, tiered alert system that could deliver the right notification and visual evidence to the users in real time.

1.2 Objectives

1. To develop an event-driven surveillance system that optimizes video storage by saving footage only during verified intrusion events detected through multi-sensor activation.
2. To design and implement an intelligent, multi-tier alert mechanism that differentiates between general motion and confirmed intrusion, thereby reducing false alarms and enhancing the reliability of user notifications.

Chapter 2. MATERIALS and METHODS

In this section, there will be the System Design, Hardware Components, and Data Storage and Alert Mechanism.

2.1 System Design and Architecture

2.1.1 Overview of System Goals

The main design aim of this surveillance system is to allow an effective, event-based recording and alerting with the help of a small, embedded platform. The system differentiates between general movement and validated intrusion and therefore only stores media when desired events occur, to minimize storage use and facilitate efficient process of evidence handling. It is based on a Raspberry Pi Zero 2W microcontroller that has sensing, computation, video, local storage, and network communication capabilities. The strategy is in line with the trends of embedded IoT surveillance, where local intelligence is given preference compared to centralized processing [1], [2].

2.1.2 Hardware and Configuration

The hardware architecture consists of motion, range and break-beam sensors that are distributed among two zones:

2.1.2.1 General Monitoring Zone:

1. PIR sensor ‘A’ (for temperature based motion detection).
2. Ultrasonic sensor (for distance measurement).

2.1.2.2 Critical Intrusion Zone:

1. PIR sensor ‘B’.
2. Microwave Doppler sensor.

3. Infrared laser transmitter and a receiver (that act as a tripwire)

The sensor are connected directly to the Raspberry Pi's General Purpose Input Output (GPIO) pins, with necessary pull-up/down configurations. The camera module (Raspberry Pi Camera V2 NoIR) is connected to Camera Serial Interface (CSI) port in order to capture still images and videos. It also features an SSD1306 Organic Light Emitting Diode (OLED) screen (Inter-Integrated Circuit (I2C) interface) to provide feedback on the system status and a piezo speaker to play audible alerts when intrusion is confirmed [3], [4].

2.1.3 Logical Tiering and Sensor Fusion Design

The system uses a hierarchical and logic-based detection model. The processing of events is done on two levels:

2.1.3.1 General Motion Detection Logic

When PIR 'A' is activated and the ultrasonic sensor reads a distance of less than 3 meters it is a motion event. This kind of two sensor verification minimizes the false positive due to external noise or heat source [5], [6]. Upon verification, the system will take a snapshot and will trigger a Discord notification that will say; 'Motion Detected', but no video is taken.

2.1.3.2 Intrusion Detection Logic

A high-confidence intrusion can only be attested when all three critical-zone sensors confirm detection, i.e., (1) PIR B detects the movement, (2) the RCWL Doppler sensor reads the movement, and (3) the laser beam is broken. This triple confirmation design is constructed to reduce false alarms and to make sure that recording and notification are only done when all sensors concur within a short period of time [7]. Such a rule-based sensor fusion model is specifically selected

instead of applying ML-based inference to ensure the system can be understood and to minimize processing overhead on the embedded processor [8].

2.1.4 Data Acquisition and Processing Pipeline

On event confirmation, operations that take place depending upon the event type are as follows:

2.1.4.1 Motion Detection (Logic 1):

1. Still photograph that was taken with ‘rpicam-still’.
2. Saved image with a time stamped name.
3. Sent a notification to Discord webhook with an image attached.

2.1.4.2 Intrusion Detection (Logic 2):

1. Snap picture and 20 seconds video taken.
2. Videos were taken with ‘rpicam-vid’ and stored locally.
3. Buzzer was turned on to generate alarm.
4. Image and video are uploaded to Google drive with rclone.
5. Sent discord notification along with both media files.

All recorded media are placed in ordered directories (/home/pi/videos/photos/ and /clips/) with the help of ISO 8601 time-stamps to make them understandable and traceable. The media filenames consist of source logic and time (e.g., TRIPLE20251028T145620Z.mp4) to allow sorting and indexing of evidence [9].

2.1.5 Live streaming and Monitoring.

The system keeps video streaming over TCP by using ‘rpicam-vid’ in listen mode. This stream may be viewed with the VLC or other Real Time Streaming Protocol (RTSP) compatible applications in order to monitor in real-time. This stream is however, short lived, it is not stored and this spares storage resources and

follows the event-driven storage philosophy [10]. This design decouples monitoring and evidence collection.

2.1.6 On-Device Display and Visual Feedback

The SSD1306 OLED shows live system status, including:

1. System state indicators (for sensor state).
2. Network IP address.
3. "System Armed" status.
4. There are flashing text alerts (MOTION DETECTED or INTRUDER!).

OLED updates are executed in a special background thread to allow uninterrupted functionality [11].

2.1.7 Alerts and Cloud Incorporation

Webhooks are used to send all alerts to Discord. These messages include:

1. Outline (Motion Detected or Intrusion Detected).
2. Snapshots that are timestamped.
3. 20s video clip (only for Logic 2).

Rclone uploads media files to the Google Drive using credentials stored in the device. The uploads use background threads so that the event detection and capture will not be affected even in case of file transfers [12].

2.1.8 Metadata and Evidentiary Integrity

In order to facilitate forensic preparedness, each media capture will be accompanied by:

1. Timestamp in file name of ISO format.
2. Metadata of the events (e.g. sensor states at trigger time)

This strategy is based on the principles of handling digital evidence to offer a minimum chain-of-custody model despite using lightweight devices [13].

2.1.9 Software Structure

The primary software is Python based, and is modular:

1. main() is used to initialize GPIO, OLED, streaming and event callbacks.
2. Sensor triggered callbacks are used to process logic and invoke 'handletrigger()'.
3. Blocking tasks such as video capture and uploads are offloaded onto threads.
4. The 'triggerlock' and 'debounceinterval' are used to avoid a repeated trigger in the period of cooldown.

Such a model of concurrency is used to make sure that the device does not become overloaded by simultaneous sensor activity, and time-sensitive alerts are not postponed [14].

2.1.10 Summary

The architecture makes use of the modular and event-driven components to develop intelligent surveillance system. The system has a reliable performance through its use of a different sensors, local video capture, cloud storage, and real-time alerts, optimizing performance in terms of storage and evidence fidelity.

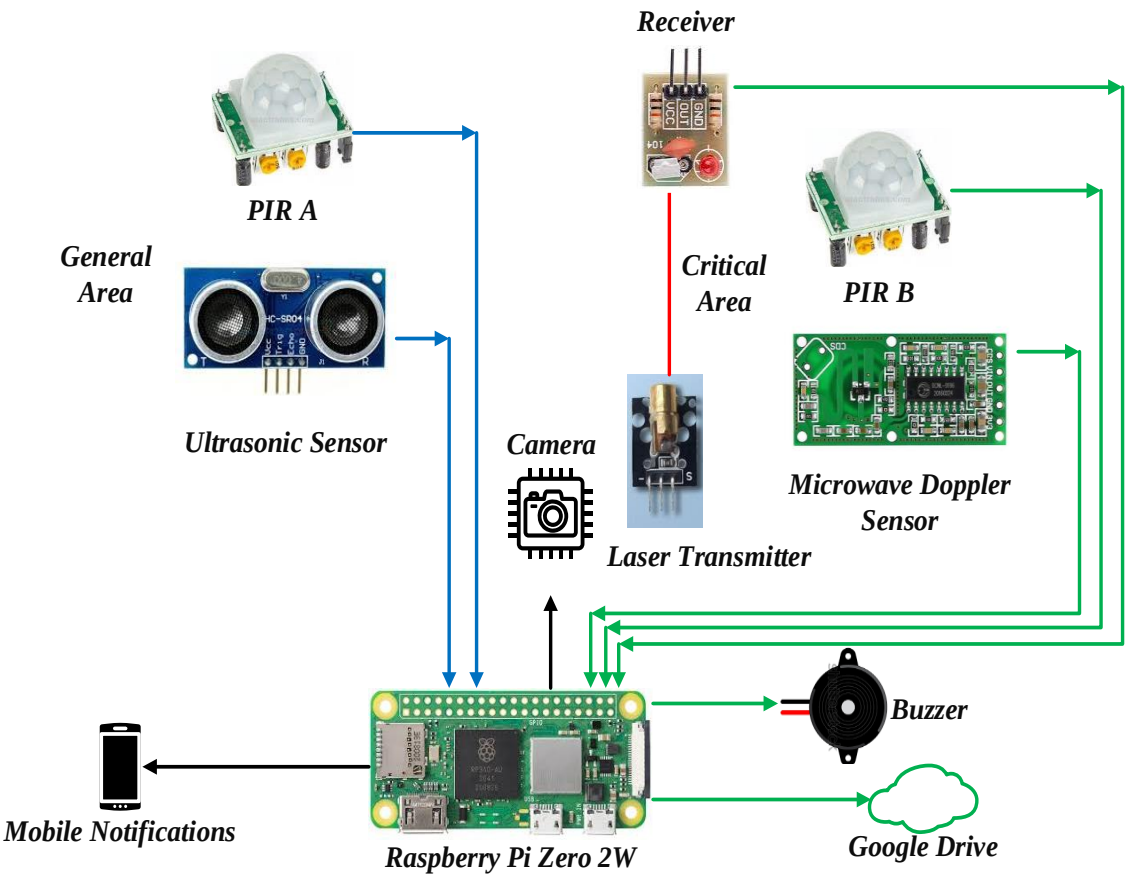


Figure 1: Project Design

2.2 Data Storage and Alert Process

2.2.1 Overview

The main feature of the proposed surveillance system is efficient and secure data storage. The architecture has made use of an event based storage model implying that only the relevant data of videos and images are stored in case a confirmed motion or intrusion event is identified. This discriminative approach will reduce the storage requirements, less load on processing, and guarantee that evidence-related footage will be stored in a traceable, time-stamped format [15], [16]. Moreover, a smart alert mechanism signals the user in real-time ensuring that the system is responsive and reliable by differentiating between general movement and intrusion [17].

2.2.2 Event-Driven Storage

Conventional surveillance systems capture video on a continuous basis, which results in huge amounts of redundant video that takes up storage capacity and makes it difficult to access them [18]. The current design substitutes this with event based capture, which is only activated when movement or intrusion is detected by multi-sensor logic. In the case of a general motion event, one still picture is taken and stored locally. In the case of intrusion event, a video clip of 20 seconds is captured along with an image and metadata of the event. All the files are automatically dated and formatted in the ISO 8601 format to ensure that they are always consistent and easy to locate [19]. The solution minimizes the data redundancy and is in line with the efficient storage practices that have been detailed in the modern embedded surveillance systems [20].

2.2.3 Local Storage Management

The media (images and video clips) is stored in the microSD card of the Raspberry Pi Zero 2W which is the local, immediate access storage. Information is arranged in a hierarchical way in directories according to type of events and date (/home/pi/storage/motion/YYYY-MM-DD/).

Such an organized method eases the process of retrieving and storing the data. SD card is formatted with FAT32 to be compatible with most operating systems and also so that it may be easily accessed manually once extracted [21], [22]. A clean-up operation periodically removes old temporary files so that it can run continuously without any human intervention [23].

2.2.4 Cloud-Based Storage (Google Drive)

The system is configured to integrate with Google drive using an open-source utility, rclone to ensure it has redundancy and can be accessed remotely. Upon detection of intrusion, the 20 seconds video and the image therein are automatically uploaded to a secure cloud folder named by the time of event. The local and cloud model of dual storage is more effective in increasing the survivability of the data even when the SD card fails or is physically tampered with [24].

File uploading is asynchronous and operates in parallel threads, so that it does not block camera or sensor use, which is in line with an effective IoT upload plan in embedded systems [25]. The access-controlled environment in Google Drive provides a secure way to keep the evidence and be able to access it only by authorized users [26].

2.2.5 Metadata and Evidence Integrity

Every saved file has minimal metadata that is kept in a small .json sidecar file that contains:

1. Timestamp of capture.
2. Event type (Motion/Intrusion).
3. Sensor states.
4. Integrity verification in file checksum (SHA-256).

These steps facilitate data provenance and authenticity, which are important in developing evidentiary reliability in forensic or administrative review procedures [27]. SHA-256 hashing is used in accordance with the best practices of digital forensics, which entails ensuring files are not compromised after capturing [28].

2.2.6 Architecture of Alerts

The alert subsystem consists of the Discord Webhooks that were selected due to their simplicity, reliability, and the capacity to accommodate multimedia attachments. Alerts are triggered with the format of a structured HTTP POST requests and include:

1. Short message about the event (“Motion Detected” or “Intrusion Detected”).
2. Timestamp of detection.
3. Media (image of motion and video and image of intrusion).

Discord servers are a simple, real-time notification system where users can be alerted by their systems on either the mobile or desktop device [29]. This solution removes a dedicated server backend and provides a complete traceability of events.

2.2.7 Distinguished Notifications

The system has a tiered alerting system which differentiates between general and critical events.

1. Tier 1 - Motion Alert: PIR and ultrasonic sensors detect motion in a general area. Produces an image with a text stating; ‘motion detected’.
2. Tier 2 - Intrusion Alert: This is activated when the PIR sensor, microwave sensor, and laser sensor are used together in the critical zone. Sends a picture and video clip along with the message stating; "Intrusion Detected".

This distinction assists users to focus on an important event and yet maintain the situational awareness by being notified of the general movement [30].

2.2.8 Live Streaming and Monitoring

The system has an active video stream that can be accessed by VLC media player over the local network in addition to event-based notifications. This stream is based on a lightweight RTSP-like application that is built around 'rpicam-vid', 'listen mode' and is independent of the alert system. Live streaming gives real-time situational awareness without using storage resources because the stream is not stored [31].

2.2.9 Fail Data Handling

Should there be a failure in cloud connectivity on an upload, the system puts files awaiting of upload in a temporary queue (/home/pi/upload_queue/). On a re-connection the upload thread will automatically restart. This will make sure that no event recordings are lost when network outages occur transiently, something that is advisable in IoT-based remote monitoring systems [32]. The local storage element is also a backup one and thus continuity is also maintained in case of a partial failure condition [33].

2.2.10 Summary of Data Flow

The general flow of information and alerts may be summarized as follows:

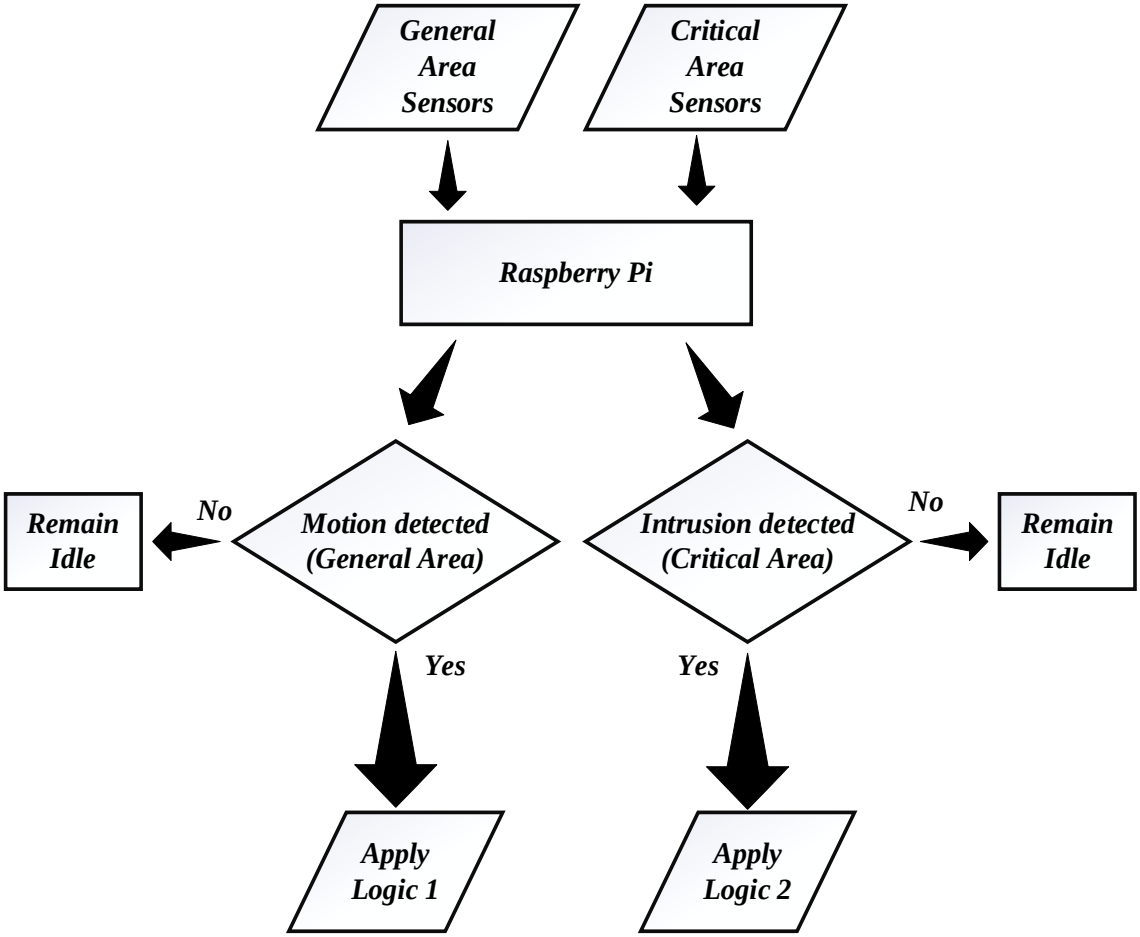
1. Sensor trigger: Motion in general area or Intrusion in a critical zone detected.
2. Capture: A snapshot and a 20 second video recorded by camera according to the logic level.
3. Local storage: Storage on microSD with metadata.
4. Alert: A message sent through Discord and a buzzer goes off in case of 'intrusion'.

5. Uploading to cloud: The video clip sent to Google Drive.
6. Queue Recovery**:** Recovery of Queues when offline.

This sequential yet non-blocking workflow balances real-time performance with data safety, embodying an event-driven, IoT-based design that is lightweight, secure, and efficient [34].

It is a non-blocking and sequential workflow that is lightweight, safe, and efficient, IoT based event driven design to balance performances in real-time [34].

Figure 2: Flow Chart



General Area Sensors:
1. PIR 1
2.Ultrasonic

Critical Area Sensors:
1. PIR 2
2.Microwave Doppler
3. Laser Beam

Logic 1:
1. Send Discord Notification 'Motion Detected'
2. Send Snapshot on Discord

Logic 2:
1. Send Discord Notification 'Intrusion Detected'
2. Send Snapshot on Discord
3. Save a 20s Clip on Local Storage and Cloud Storage
4, Sound the Buzzer

Chapter 3. Results and Discussion

This section includes the results of our surveillance system. To give a clear indication of storage optimization, we compare our event-based surveillance model with theoretically calculated storage requirements for continuous recording. The theoretical calculation for continuous recording is done with camera bitrate of 2 Mbps. Only the detection of relevant activity is registered by our event-based model. This is the best way to optimize storage since idle time is not recorded.

3.1 Test setup

Table 2: Test setup

Test Item	**Value / Details**
Video capture bitrate (for clips & continuous recording)	2,000,000 bits/s (2 Mbps)
Intrusion clip length	20 seconds per clip
Snapshot quality	JPEG quality ~50 ~150 Kilobyte (KB) typical per snapshot
Device	Raspberry Pi Zero 2W, SD card (16 GB), NoIR camera, Motion sensors

3.2 Theoretically Calculated Continuous Recording

1. Continuous bitrate = 2,000,000 bits per second.
2. Seconds per hour = 3,600.
3. Bits per hour = 2,000,000 × 3,600 = 7,200,000,000 bits.
4. Bytes per hour = 7,200,000,000 ÷ 8 = 900,000,000 bytes.
5. Storage per hour ≈ **900 MB** (decimal).

Table 3 : Continuous Recording

Item	Value
Bitrate	2,000,000 bits/s
Seconds per hour	3,600 s
Bits per hour	7,200,000,000 bits
Bytes per hour	900,000,000 bytes
Storage per hour (approx.)	900 MB

3.3 Event-based Recording

This system only captures the general area motion snapshot and intrusion snapshot and 20s video clip. The amount of stored bytes is based on the number of identified events.

Table 4: File Type and Size

Event type	File stored	Size per file
Motion (tier 1)	Snapshot	150 KB
Intrusion (tier 2)	20 s video clip + snapshot	5 MB + 150 KB

Table 5: Event Summary

Event	Ground truth	Detected?	Stored file(s)	Size
1	Motion	Yes	Snapshot	150 KB
2	Motion	Yes	Snapshot	150 KB
3	Intrusion	Yes	Clip & snapshot	5 MB + 150 KB
4	Motion	No	n/a	0 KB
5	Intrusion	Yes	Clip & snapshot	5 MB + 150 KB

6	Motion	Yes	Snapshot	150 KB
7	Intrusion	Yes	Clip & snapshot	5 MB + 150 KB
8	Motion	Yes	Snapshot	150 KB
9	Motion	Yes	Snapshot	150 KB
10	Intrusion	Yes	Clip & snapshot	5 MB + 150 KB

3.3.1 Calculations

1. Snapshots stored: 5 (from detected motions) × 150 KB = 750 KB.
2. Clips stored: 4 × 5 MB = 20 MB.
3. Total stored ≈ 20.75 MB.

Table 6: Storage Comparison

Metric	**Continuous (per hour)**	**Event-based (10 events)**
Stored media	900 MB / hour (theoretical)	20.75 MB
Number of stored files	Continuous stream segments	9 files (5 snapshots + 4 clips)
Detection accuracy	n/a	9 of 10 = 90%

3.4 Locating Relevant Evidence

Finding the relevant events in continuous video is time consuming, which involves scrubbing through hours of the footage. In this system, the alerts are created with time stamp of events, thus retrieving them timely. Video clips or snapshots associated with the identified events are instantly available to the reviewers, which normally saves a lot of search time.

3.5 Cloud Upload and Network Usage

Continuous recording continuously feeds data to the local storage or to the cloud which consumes a lot of bandwidth. Event based recording, on the other hand, will only send in snapshots or short clips that are relevant. This will reduce network load and costs in cloud storage. The uploaded data volume is directly proportional to the amount of events detected and not the time period, which makes it bandwidth-efficient.

Table 7: Network and Cloud usage Comparison

Metric	**Continuous**	**Event-Based**
Network load	High/constant	Low/variable
Upload frequency	Continuous	On event trigger
Cloud cost	High	Optimized

3.6 Discussions

3.6.1 General Motion Event

1. The system captures only a snapshot in case of a general motion event .
2. They are easy to review.
3. Storage requirement only grows with the number of motion events.

3.6.2 Intrusion Event

1. The intrusions result in a 20 s video clip and snapshot.
2. Clips are created only in case of confirmed intrusion.
3. Clips are more evidently valuable.

3.6.3 Observations

1. The system works effectively.

2. The system effectively identifies the motion and intrusion.
3. Event based recordings only store relevant activity.
4. Storage utilized is dependent on the number of events that take place.
5. Event based recording does not need to retain extensive idle video as compared to continuous recording.
6. There are occasional false triggers which arise due to slight background movements or even change in lighting but these are minimal.
7. It is easier to do post-event analysis because of clarity and time stamping of images.
8. Event-based files are small and can be managed in either local or remote monitoring.
9. It becomes easier to check the corresponding footage and decreases SD card writes.

3.7 Recommendations

1. Use adaptive thresholding to reduce false positives with changing light conditions.
2. Use better sensors or infrared modules in low-light conditions.
3. Include the ability to distinguish certain objects or human gestures.
4. Adapt sensor timings for more noisy environments.

3.8 Conclusion

The event based surveillance system effectively reduces storage space and ensures a reliable detection accuracy. The system gives quicker access to related evidence, an optimum bandwidth consumption, and an optimized storage than continuous recording. The system can be used as a viable and sustainable way of

surveillance with some enhancements on environmental adaptability and classification features.

References

[1] A. Karbalaie, “Event detection in surveillance videos: A review,” *Multimed. Tools Appl.*, 2022.

[2] A. Al-Fuqaha, “Internet of Things: A Survey on Enabling Technologies,” *IEEE Communications Surveys & Tutorials*, 2015.

[3] R. P. Foundation, “GPIO Pinout Guide,” 2024, [Online]. Available: https://www.raspberrypi.org/documentation/

[4] A. Industries, “SSD1306 OLED Display Hookup Guide,” 2023, [Online]. Available: https://learn.adafruit.com/ssd1306

[5] A. Shokrollahi, “Passive Infrared Sensor-Based Occupancy Monitoring,” *MDPI Sensors*, 2024.

[6] K. Panda, “Environmental impacts on ultrasonic sensor accuracy,” *AIP Conf. Proc.*, 2016.

[7] L. Manchineella, “Motion Detection Using Microwave Radar Sensor (RCWL-0516),” *SSRN*, 2021.

[8] P. Singh, “Rule-based sensor fusion for embedded surveillance,” *IEEE Embed. Syst. Lett.*, 2021.

[9] B. Guttman, “Digital Evidence Preservation,” 2022.

[10] V. Patel, “Hybrid streaming and event recording architecture,” *Smart IoT Systems Journal*, 2022.

[11] A. Industries, “Using OLED with Raspberry Pi (SSD1306),” 2023, [Online]. Available: https://learn.adafruit.com/ssd1306-on-raspberry-pi

[12] R. contributors, “rclone: Cloud storage synchronization tool,” 2024, [Online]. Available: https://rclone.org/docs/

[13] SWGDE, “Best Practices for Digital Video Authentication,” 2023.

[14] P. O’Connor and M. J. Roberts, “Concurrency Patterns in Embedded Surveillance Systems,” *Embedded Computing Journal*, 2020.

[15] M. Rahman and T. Nguyen, “Event-triggered recording for low-bandwidth surveillance,” *IEEE Internet Things J.*, 2021.

[16] A. Singh and R. Gupta, “Optimized storage models for IoT surveillance systems,” *MDPI Sensors*, 2022.

[17] F. Wei, “Storage-optimized IoT surveillance design,” *IEEE Access*, 2023.

[18] D. Park, “Analysis of continuous vs. event-driven surveillance recording,” *IEEE Smart Systems Journal*, 2020.

[19] SWGDE, “Best Practices for Digital Video Authentication,” 2023.

[20] M. Chen, “Efficient data retention in embedded monitoring systems,” *Journal of IoT Applications*, 2022.

[21] S. Corp., “microSDHC Card Product Specification,” 2022.

[22] R. P. Foundation, “File System and Data Management on Raspberry Pi,” 2024.

[23] L. Sharma, “Automated file management for long-term embedded systems,” *Embedded Systems Letters*, 2023.

[24] Rclone.org, “rclone: Cloud storage synchronization tool,” 2024, [Online]. Available: https://rclone.org/docs/

[25] T. Nguyen, “Concurrent upload optimization in IoT surveillance,” *IEEE Embed. Syst. Lett.*, 2021.

[26] G. Cloud, “Google Drive Security and Privacy Whitepaper,” 2023.

[27] NIST, “Digital Evidence Preservation Framework (NIST IR 8387),” 2022.

[28] S. Garfinkel, “Integrity verification in digital forensics,” *Digit. Investig.*, 2021.

[29] D. Inc., “Using Webhooks for Real-Time Integration,” 2024.

[30] P. Singh, “Rule-based sensor fusion for embedded surveillance,” 2021.

[31] V. Patel, “Hybrid streaming and event recording architecture,” *Smart IoT Systems Journal*, 2022.

[32] R. Malik, “Fault-tolerant upload mechanisms in IoT devices,” *IEEE Access*, 2022.

[33] A. Nadim and T. Reddy, “Multi-sensor fusion in low-cost surveillance nodes,” *Smart Systems Review*, 2024.

[34] A. Karbalaie, “Event detection in surveillance videos: A review,” *Multimed. Tools Appl.*, 2022.

Appendix

```
import RPi.GPIO as GPIO
import time
import subprocess
import threading
import requests
import json
import socket
from pathlib import Path

# --- OLED IMPORTS ---
import board
import busio
from PIL import Image, ImageDraw, ImageFont
import adafruit_ssd1306

# --- CONFIG ---
# !!! UPDATE THESE VARIABLES !!!
DISCORD_WEBHOOK_URL =
'https://discordapp.com/api/webhooks/1424340105317060623/o42AvQgBy8FC5b
6espYW6pMeaftJWNW4zx9GMttOkAyyJmHWEp_YNuvQhMQfuJMDyJCQ'
RCLONE_REMOTE = 'camsetup:pi-cam'
CLIP_SECONDS = 20
JPEG_QUALITY = 50
VIDEO_BITRATE = 2000000

# STREAMING CONFIG (Must match your Pi's IP address)
STREAM_PORT = 5000

# --- PIN MAPPING (Based on Final Wiring) ---
# Inputs
PIR_A_PIN = 17      # PIR used for Ultrasonic confirmation (Logic 1)
PIR_B_PIN = 27      # PIR used for Laser/RCWL confirmation (Logic 2)
LASER_RX_PIN = 22   # Single Laser Receiver Pin
RCWL_PIN = 4        # RCWL Doppler Radar Input (Logic 2)
TRIG_PIN = 23       # HC-SR04 Trigger
ECHO_PIN = 24       # HC-SR04 Echo

# Outputs
BUZZER_PIN = 18
ARM_LED = 26        # ARM_LED moved to BCM 26
LASER_TX_PIN = 5    # HW-493 Laser Transmitter Control

# OLED Configuration
OLED_WIDTH = 128
OLED_HEIGHT = 64
FONT_PATH = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf"
FONT_SIZE_SMALL = 10
```

```
FONT_SIZE_ALERT = 14

# --- GLOBAL STATE ---
trigger_lock = threading.Lock()
stream_process = None # Holds the continuous stream process
last_trigger_time = 0
debounce_interval = 10
ultrasonic_triggered = False
oled = None
oled_thread_active = True


# --- PATHS ---
MEDIA_DIR = Path('/home/pi/videos')
PHOTO_DIR = MEDIA_DIR / 'photos'
VIDEO_DIR = MEDIA_DIR / 'clips'
PHOTO_DIR.mkdir(parents=True, exist_ok=True)
VIDEO_DIR.mkdir(parents=True, exist_ok=True)


# --- HELPERS ---

def get_ip_address():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except:
        return "No IP"

def set_arm_led(state):
    """Controls the ARM status LED and Laser TX module."""
    GPIO.output(ARM_LED, state)
    GPIO.output(LASER_TX_PIN, state) # Turn Laser ON/OFF with ARM LED

def activate_buzzer(duration=0.5, cycles=3):
    """Blinks the buzzer for critical alerts."""
    try:
        for _ in range(cycles):
            GPIO.output(BUZZER_PIN, GPIO.HIGH)
            time.sleep(duration / (2 * cycles))
            GPIO.output(BUZZER_PIN, GPIO.LOW)
            time.sleep(duration / (2 * cycles))
    except Exception as e:
        print(f"Buzzer error: {e}")

# --- OLED FUNCTIONS ---
```

```
def oled_thread_task():
    """Constantly updates the OLED with system status and IP."""
    global oled_thread_active
    ip_address = get_ip_address()

    try:
        font_small = ImageFont.truetype(FONT_PATH, FONT_SIZE_SMALL)
    except IOError:
        font_small = ImageFont.load_default()

    while oled_thread_active:
        if not oled:
            time.sleep(1)
            continue

        try:
            image = Image.new('1', (OLED_WIDTH, OLED_HEIGHT), 0)
            draw = ImageDraw.Draw(image)

            # Status
            draw.text((0, 0), "SYSTEM ARMED", font=font_small, fill=255)
            draw.text((0, 12), "--------------------", font=font_small, fill=255)

            # Logic Status (Sensor states)
                draw.text((0, 28), f"L1 (PIR A): {GPIO.input(PIR_A_PIN)}",
font=font_small, fill=255)
                draw.text((0, 38), f"L2 (TRIPLE): {GPIO.input(PIR_B_PIN)}
{GPIO.input(RCWL_PIN)} {GPIO.input(LASER_RX_PIN)}", font=font_small,
fill=255)

            # IP Address
            draw.text((0, 52), f"IP: {ip_address}", font=font_small, fill=255)

            oled.image(image)
            oled.show()
            time.sleep(1) # Update every 1 second

        except Exception as e:
            print(f"OLED background update error: {e}")
            time.sleep(5)


def display_alert(message, duration=5):
    """Displays a blinking, bold alert on the OLED, overriding the background
thread."""
    global oled_thread_active

    # Temporarily pause background updates
    oled_thread_active = False
```

```
        try:
            font_alert = ImageFont.truetype(FONT_PATH, FONT_SIZE_ALERT)
        except IOError:
            font_alert = ImageFont.load_default()

        start_time = time.time()

        while (time.time() - start_time) < duration:
            if not oled:
                break

            for fill_state in [255, 0]: # Blink on/off
                image = Image.new('1', (OLED_WIDTH, OLED_HEIGHT), 0)
                draw = ImageDraw.Draw(image)

                # Center the text (rough estimate for 128x64)
                draw.text((0, 20), message, font=font_alert, fill=fill_state)

                oled.image(image)
                oled.show()
                time.sleep(0.3)
                if (time.time() - start_time) >= duration:
                    break

        # Clear and resume background updates
        oled.fill(0)
        oled.show()
        oled_thread_active = True


    # --- STREAMING & CAMERA FUNCTIONS ---

    def start_live_stream():
        """Starts the continuous video stream over LAN using rpicam-vid and a TCP
socket."""
        global stream_process

        if stream_process is None:
            print(f"Starting continuous stream on port {STREAM_PORT}...")
            try:
                # rpicam-vid command to capture video and output to a TCP socket
                stream_command = [
                    'rpicam-vid', '-t', '0', '--inline', '-n',
                    '--listen', '-o', 'tcp://0.0.0.0:' + str(STREAM_PORT),
                    '--width', '640', '--height', '480'
                ]

                stream_process = subprocess.Popen(stream_command)
```

```
                                print(f"Stream running. View via VLC: tcp://<Pi_IP_Address>:{STREAM_PORT}")

        except Exception as e:
            print(f"Error starting stream: {e}")

def stop_live_stream():
    """Stops the continuous stream process."""
    global stream_process
    if stream_process:
        print("Stopping continuous stream...")
        stream_process.terminate()
        stream_process.wait()
        stream_process = None
        print("Stream stopped.")


def capture_photo(path: Path):
    """Captures a still image using rpicam-still."""
    print(f"Capturing photo to: {path}")
    try:
                subprocess.run(['rpicam-still', '-n', '-o', str(path), '--quality', str(JPEG_QUALITY)], check=True)
        print("Capture complete.")
        return True
    except Exception as e:
        print(f'Capture photo error: {e}')
        return False

def capture_video(path: Path):
    """Captures a video clip using rpicam-vid."""
    print(f"Recording video to: {path}")
    try:
        subprocess.run([
            'rpicam-vid',
            '-t', str(CLIP_SECONDS * 1000),
            '-o', str(path),
            '--bitrate', str(VIDEO_BITRATE),
            '--hflip', '--vflip', '-n' # Add hflip/vflip if needed
        ], check=True)
        print("Video recording complete.")
        return True
    except Exception as e:
        print(f'Video recording error: {e}')
        return False

def upload_with_rclone(file_path: Path):
    """Uploads a file to Google Drive using the rclone subprocess."""
    print(f"Starting rclone upload of: {file_path.name}")
    try:
```

```
            subprocess.run(['rclone', 'copy', str(file_path), RCLONE_REMOTE],
check=True)
        print(f"Upload Done for {file_path.name}.")
        # Optional: Delete local file after successful upload
        # file_path.unlink()
        return True
    except subprocess.CalledProcessError as e:
        print(f'Rclone upload failed: {e}')
        return False
    except Exception as e:
        print(f'General upload error: {e}')
        return False

def send_webhook_alert(content, file_path=None):
    """Sends a message or file to the Discord webhook."""
                                if                DISCORD_WEBHOOK_URL                 ==
'YOUR_ACTUAL_DISCORD_WEBHOOK_URL_HERE':
        print("Webhook alert skipped: URL not set.")
        return

    payload = {'content': content}
    files = {}

    if file_path and Path(file_path).exists():
        file_name = Path(file_path).name
        files['file'] = (file_name, open(file_path, 'rb'))

    try:
        print(f"Sending alert: {content}")
        if files:
            response = requests.post(DISCORD_WEBHOOK_URL, data=payload,
files=files, timeout=30)
        else:
            response = requests.post(DISCORD_WEBHOOK_URL, json=payload,
timeout=10)

        response.raise_for_status()
        print(f"Discord alert sent successfully. Status: {response.status_code}")
        if files:
            files['file'][1].close()

    except requests.exceptions.RequestException as e:
        print(f"Discord webhook error: {e}")
        if files:
            files['file'][1].close()


# --- TRIGGER HANDLER ---

def handle_trigger(source, record_video=False, upload=False):
```

```
        global last_trigger_time

        with trigger_lock:
            current_time = time.time()
            if current_time - last_trigger_time < debounce_interval:
                print(f"Debounce active. Ignoring trigger from {source}.")
                return

            last_trigger_time = current_time
            print(f"!!! ALERT !!! Trigger received from {source}. Actions starting...")

            # 1. Capture snapshot immediately
            timestamp = time.strftime('%Y%m%d_%H%M%S')
            photo_path = PHOTO_DIR / f'snapshot_{timestamp}.jpg'

            if capture_photo(photo_path):
                alert_text = f"ðŸš¨ {source} detected! Snapshot attached."
                send_webhook_alert(alert_text, file_path=photo_path)

                # 2. Handle video and upload if required (Logic 2)
                if record_video:
                    activate_buzzer(duration=1.5, cycles=5)
                    video_path = VIDEO_DIR / f'clip_{timestamp}.mp4'

                    # Display critical alert on OLED
                    display_alert("INTRUDER!", CLIP_SECONDS + 5)

                    if capture_video(video_path):
                        send_webhook_alert(f"ðŸ“½ï¸• Video clip ({CLIP_SECONDS}s)
recorded.", file_path=video_path)

                        if upload:
                            # Upload both photo and video in separate threads
                            threading.Thread(target=upload_with_rclone, args=(photo_path,),
daemon=True).start()
                            threading.Thread(target=upload_with_rclone, args=(video_path,),
daemon=True).start()
                    else:
                        send_webhook_alert("â• Œ Video recording FAILED.")

    # --- SENSOR CALLBACKS ---

    def get_distance():
        """Measures distance using the HC-SR04 ultrasonic sensor."""
        GPIO.output(TRIG_PIN, GPIO.LOW)
        time.sleep(0.000002)
        GPIO.output(TRIG_PIN, GPIO.HIGH)
        time.sleep(0.000010)
        GPIO.output(TRIG_PIN, GPIO.LOW)
```

```
        pulse_start = time.time()
        pulse_end = time.time()

        # Wait for ECHO to go HIGH (Start of pulse)
        while GPIO.input(ECHO_PIN) == GPIO.LOW:
            pulse_start = time.time()
            if pulse_start - pulse_end > 0.1: # Timeout
                return 999

        # Wait for ECHO to go LOW (End of pulse)
        while GPIO.input(ECHO_PIN) == GPIO.HIGH:
            pulse_end = time.time()
            if pulse_end - pulse_start > 0.1: # Timeout
                return 999

        duration = pulse_end - pulse_start
        # Speed of sound is 34300 cm/s
        distance = duration * 17150 # duration * (34300 / 2)

        return round(distance / 100, 2) # Return in meters

    def ultrasonic_check():
        """Puts the ultrasonic_triggered flag up if distance is < 3m."""
        global ultrasonic_triggered
        try:
            distance = get_distance()
            if distance < 3.0 and distance > 0: # Trigger if closer than 3 meters
                ultrasonic_triggered = True
            else:
                ultrasonic_triggered = False
        except Exception as e:
            print(f"Ultrasonic error: {e}")
            ultrasonic_triggered = False


    # LOGIC 1: Ultrasonic (<3m) + PIR A (17) -> Snapshot & Notify
    def pir_a_cb(channel):
        """Logic 1: PIR A confirms Ultrasonic reading."""
        global ultrasonic_triggered
        # Check if PIR A is rising AND the ultrasonic flag is currently TRUE
        if GPIO.input(PIR_A_PIN) == GPIO.HIGH and ultrasonic_triggered:
            print("LOGIC 1 Triggered: Ultrasonic + PIR A CONFIRMED")
            display_alert("MOTION DETECTED", 3)
                handle_trigger('USONIC_PIR_A_CONFIRM',   record_video=False,
upload=False)
            ultrasonic_triggered = False # Reset flag

    # LOGIC 2: PIR B (27) + Laser Break (22) + RCWL (4) -> Record, Upload,
Stream
    def high_security_cb(channel):
```

```
        """Logic 2: Triple confirmation check."""
        # This function is triggered ONLY by the Laser beam break (FALLING edge)
        if channel == LASER_RX_PIN and GPIO.input(LASER_RX_PIN) == 0:

            # Check the state of the two other sensors (PIR B and RCWL)
            pir_b_state = GPIO.input(PIR_B_PIN)
            rcwl_state = GPIO.input(RCWL_PIN)

            # All three must be TRUE (Laser break (0), PIR B (1), and RCWL (1))
            if pir_b_state and rcwl_state:
                print("LOGIC 2 Triggered: PIR B + Laser + RCWL CONFIRMED")
                # Stream is running continuously, so we only handle the recording/alerts
                    handle_trigger('TRIPLE_SECURITY_TRIP',  record_video=True,
upload=True)
            else:
                    print(f"Triple check failed: Laser ok, but PIR B={pir_b_state},
RCWL={rcwl_state}")


    # --- MAIN ---

    def main():
        global oled

        try:
            # 1. HARDWARE SETUP
            GPIO.setmode(GPIO.BCM)
            # Inputs
            GPIO.setup(PIR_A_PIN, GPIO.IN)
            GPIO.setup(PIR_B_PIN, GPIO.IN)
            GPIO.setup(RCWL_PIN, GPIO.IN)
            GPIO.setup(LASER_RX_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
# Receiver
            GPIO.setup(ECHO_PIN, GPIO.IN)
            # Outputs
            GPIO.setup(TRIG_PIN, GPIO.OUT)
            GPIO.setup(BUZZER_PIN, GPIO.OUT)
            GPIO.setup(ARM_LED, GPIO.OUT)
            GPIO.setup(LASER_TX_PIN, GPIO.OUT) # Transmitter

            # Initial States (System Armed)
            set_arm_led(GPIO.HIGH) # Turns LED and Laser TX ON
            GPIO.output(BUZZER_PIN, GPIO.LOW)
            GPIO.output(TRIG_PIN, GPIO.LOW) # Ensure TRIG is low initially

            # 2. OLED SETUP
            i2c = busio.I2C(board.SCL, board.SDA)
            oled = adafruit_ssd1306.SSD1306_I2C(OLED_WIDTH, OLED_HEIGHT,
i2c)
            oled.fill(0)
```

```
        oled.show()

        # 3. START CONTINUOUS LIVE STREAM & OLED THREADS
        start_live_stream()
        threading.Thread(target=oled_thread_task, daemon=True).start()

        # 4. EVENT DETECTION
    GPIO.add_event_detect(PIR_A_PIN, GPIO.RISING, bouncetime=800)
      GPIO.add_event_callback(PIR_A_PIN, pir_a_cb)

                GPIO.add_event_detect(LASER_RX_PIN,      GPIO.FALLING,
bouncetime=400)
        GPIO.add_event_callback(LASER_RX_PIN, high_security_cb)

        # 5. ULTRASONIC LOOP
        while True:
            # Check ultrasonic distance every 0.5s
            ultrasonic_check()
            time.sleep(0.5)

    except KeyboardInterrupt:
        print("Exiting program.")
    except Exception as e:
        print(f"An unhandled error occurred: {e}")
    finally:
        global oled_thread_active
        oled_thread_active = False # Stop OLED thread
        if oled:
            oled.fill(0)
            oled.show()
        stop_live_stream() # Stop the stream
        set_arm_led(GPIO.LOW) # Turn off LED and Laser TX
        GPIO.cleanup()

if __name__ == '__main__':
    main()
```